



Taller de Comandos Linux

Juan Auz, Quillupangui Ana María, Palomeque Matías, Lanchimba Danny

2026-06-19

1 Objetivos

- Conocer el funcionamiento del terminal de *Linux*
- Aprender a manejar comandos del terminal de *Linux*

2 Instrucciones

Este taller/tutorial de comandos de Linux tiene por objetivo ejercitar en la aplicación de comandos estudiados en clase. Para esto siga las Instrucciones indicadas en cada uno de los numerales. Ejecute los ejercicios llenando los comandos adecuados dentro de los bloques de código de Emacs Org. Los resultados de la ejecución se obtienen haciendo `C-c C-c`

Para todos los ejercicios se sugiere usar directamente un distro de *Linux* o el *WSL* configurado para la clase de *Arquitectura de Computadores*. Ejecute los comandos en la terminal de Linux. Copie los comandos usados a este archivo.

Si se ha utilizado IA para consultar temas referentes al desarrollo de código en el punto 4, adjunte el archivo `.md` con el reporte de los diferentes prompts y respuestas dadas por la IA.

2.1 Ejecución desde Emacs

Si desea ejecutar los comandos desde Emacs debe observar que los comandos del `shell` de Linux se ejecutarán desde la ubicación del archivo `.org`. Por tanto, controle la ubicación de `home` en los bloques de código. Para ejecutar los bloques de código ejecute: `C-c C-c`.

3 Comandos de Linux

Para este taller descargaremos los datos del *Fashion-Mnist*, un dataset popular para la prueba de algoritmos de machine learning. Al final, el dataset quedará organizado para ser aprovechado para entrenar un modelo.

1. Utilice `cd` para localizarse en la posición de `home` de su usuario y verifique ejecutando el comando `pwd`

```
cd
pwd
```

```
/home/juand
```

2. Cree tres directorios denominados `FashionData`, `Train` y `Test`. Verifique usando el comando `ls`

```
cd ..
mkdir FashionData
mkdir Train
mkdir Test
ls
```

```
FashionData
FormatoTareas
images
Proyecto
Talleres
Test
Train
```

3. Investigue sobre el comando `wget` y utilícelo para descargar en `~/FashionData` los datos de train y test del `/Fashion-Mnist` desde el siguiente enlace: `fashion-mnist`

- (a) Imágenes de entrenamiento:
`http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-images-idx3-ubyte.gz`
- (b) Etiquetas de entrenamiento:
`http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-labels-idx1-ubyte.gz`
- (c) Imágenes de prueba:
`http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-images-idx3-ubyte.gz`
- (d) Etiquetas de prueba:
`http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-labels-idx1-ubyte.gz`

```
cd ..
cd FashionData
wget http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-images-idx3-ubyte.gz
wget http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-labels-idx1-ubyte.gz
wget http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-images-idx3-ubyte.gz
wget http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-labels-idx1-ubyte.gz
pwd
ls
```

```
/home/juand/Documentos/ArquitecturaComp/ArquitecturaComputadores/FashionData
t10k-images-idx3-ubyte.gz
t10k-labels-idx1-ubyte.gz
Test
Train
train-images-idx3-ubyte.gz
train-labels-idx1-ubyte.gz
```

4. Utilice el comando `mv` de manera recursiva para pasar los datos de entrenamiento a la carpeta `~/Train`

```
cd ..
mv FashionData/train* Train/
cd Train
ls
```

```
train-images-idx3-ubyte
train-images-idx3-ubyte.gz
train-labels-idx1-ubyte
train-labels-idx1-ubyte.gz
```

5. Utilice el comando `mv` de manera recursiva para pasar los datos de prueba a la carpeta `~/Test`

```
cd ..
mv FashionData/t10k* Test/
cd Test
ls
```

```
t10k-images-idx3-ubyte
t10k-images-idx3-ubyte.gz
t10k-labels-idx1-ubyte
t10k-labels-idx1-ubyte.gz
```

6. Verifique que la carpeta `FashionData` está vacía

```
cd ..
cd FashionData
ls
```

```
Test
Train
```

7. Los archivos anteriores están comprimidos. Investigue como descomprimir usando el comando `gunzip`. Si es necesario realice la instalación del comando. Descomprima los archivos en sus respectivas carpetas. Apunte el comando utilizado para la descompresión.

```
cd ..
gunzip -r Test/
gunzip -r Train/
cd Test/
ls
cd ..
cd Train/
ls
```

8. Mueva recursivamente las carpetas `~/Train` y `~/Test` dentro de `~/FashionData`

```

t10k-images-idx3-ubyte
t10k-images-idx3-ubyte.gz
t10k-labels-idx1-ubyte
t10k-labels-idx1-ubyte.gz
train-images-idx3-ubyte
train-images-idx3-ubyte.gz
train-labels-idx1-ubyte
train-labels-idx1-ubyte.gz

```

```

cd ..
mv Test/ FashionData/
mv Train/ FashionData/
cd FashionData/
pwd
ls

```

```

/home/juand/Documentos/ArquitecturaComp/ArquitecturaComputadores/FashionData
Test
Train

```

9. Verifique la estructura de la carpeta `/FashionData` usando el comando `~tree`

```

cd ..
cd FashionData
tree

```

```

.
├── Test
│   ├── t10k-images-idx3-ubyte
│   └── t10k-labels-idx1-ubyte
└── Train
    ├── train-images-idx3-ubyte
    └── train-labels-idx1-ubyte

```

3 directories, 4 files

4 Problema

1 Se desea realizar un registro climatológico de la ciudad de Quito. Para esto, escriba un script de Python/Java que permita obtener datos climatológicos desde el API de openweathermap. Considere la latitud y la longitud de Quito como (-0.2299, -78.5249), respectivamente. Los resultados obtenidos de la consulta al API se escriben en un archivo *clima-quito-hoy.csv*. Cada ejecución del script debe almacenar nuevos datos en el archivo. Investigue sobre **crontab** y utilícelo para obtener datos del API de *openweathermap* cada 5 minutos mediante la ejecución de un archivo ejecutable denominado *get-weather.sh*. Verifique los resultados. Todas las operaciones se realizan en Linux o en el WSL. Las etapas del problema se subdividen en:

1. Crear su API gratuito en openweathermap
2. Escribir un script en Python/Java que realice la consulta al API y escriba los resultados en *clima-quito-hoy.csv*

```

import requests
import json
import csv
import os
import pathlib
from datetime import datetime

API_KEY = 'b794b9218f841b4d649bce12eacc5aed'
LAT = -0.2299
LON = -78.5249
URL = f"https://api.openweathermap.org/data/2.5/weather?lat={LAT}&lon={LON}&appid={API_KEY}&units=m"
OUTPUT_FILE = pathlib.Path(__file__).parent / 'clima-quito-hoy.csv'

def main():
    response = requests.get(URL)
    data = response.json()
    timestamp = datetime.now().isoformat()

    existe = os.path.isfile(OUTPUT_FILE)

    with open(OUTPUT_FILE, 'a', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)
        if not existe:
            writer.writerow(['timestamp', 'data'])
        writer.writerow([timestamp, json.dumps(data)])

    print(f"[{timestamp}] Guardado - temp: {data['main']['temp']}°C")

if __name__ == '__main__':
    main()

```

None

3. Desarrollar un ejecutable *get-weather.sh* para ejecutar el programa Python/Java.

```

sudo nano get-weather.sh

#!/bin/bash
/home/juand/miniforge3/envs/iccd332/bin/python /home/juand/Documentos/ArquitecturaComp/ArquitecturaComp/Proyecto/get-weather.py

chmod +x get-weather.sh
./get-weather.sh

```

[2026-06-21T20:42:30.553918] Guardado - temp: 10.64°C

4. Configurar Crontab para la adquisición de datos. Escriba el comando configurado. `* /5 * * * * /home/juand/Documentos/ArquitecturaComp/ArquitecturaComputadores/Proyecto/get-weather.sh » /home/juand/Documentos/ArquitecturaComp/ArquitecturaComputadores/Proyecto/cron.log 2>&1`